Not All Thoughts are Generated Equal: Efficient LLM Reasoning via Multi-Turn Reinforcement Learning

Yansong Ning[1]*, Wei Li[2], Jun Fang[2], Naiqiang Tan[2], Hao Liu[1,3]†
[1] AI Thrust, The Hong Kong University of Science and Technology (Guangzhou)
[2] Didichuxing Co. Ltd
[3] CSE, The Hong Kong University of Science and Technology
yning092@connect.hkust-gz.edu.cn,
{peterliwei,fangjun,tannaiqiang}@didiglobal.com
liuh@ust.hk

Abstract

Compressing long chain-of-thought (CoT) from large language models (LLMs) is an emerging strategy to improve the reasoning efficiency of LLMs. Despite its promising benefits, existing studies equally compress all thoughts within a long CoT, hindering more concise and effective reasoning. To this end, we first investigate the importance of different thoughts by examining their effectiveness and efficiency in contributing to reasoning through automatic long CoT chunking and Monte Carlo rollouts. Building upon the insights, we propose a theoretically bounded metric to jointly measure the effectiveness and efficiency of different thoughts. We then propose **Long⊗Short**, an efficient reasoning framework that enables two LLMs to collaboratively solve the problem: a long-thought LLM for more effectively generating important thoughts, while a short-thought LLM for efficiently generating remaining thoughts. Specifically, we begin by synthesizing a small amount of cold-start data to fine-tune LLMs for long-thought and short-thought reasoning styles, respectively. Furthermore, we propose a synergizing-oriented multi-turn reinforcement learning, focusing on the model self-evolution and collaboration between long-thought and short-thought LLMs. Experimental results show that our method enables Qwen2.5-7B and Llama3.1-8B to **achieve comparable performance** compared to DeepSeek-R1-Distill-Qwen-7B and DeepSeek-R1-Distill-Llama-8B, while **reducing token length by over 80%** across the MATH500, AIME24/25, AMC23, and GPQA Diamond benchmarks. Our data and code are available at https://github.com/usail-hkust/LongShort.

1 Introduction

Long Chain-of-Thought (CoT) reasoning aims to achieve test-time scaling [1, 2] by increasing the length of the CoT [3] reasoning process. Recently, with the advancement of LLMs, such as OpenAI o-1 [4] and DeepSeek-R1 [5], long CoT reasoning has demonstrated significant improvements on various challenging tasks, such as mathematics Olympiad [6], professional coding [7] and scientific reasoning [8] and so on. Long CoT reasoning has gradually become a key capacity of LLMs. Despite these advances, the excessive token length [9] of long CoT reasoning remains a major bottleneck, limiting its effectiveness and practical applicability.

In recent literature, considerable efforts have been made to compress reasoning token length while maintaining performance [10]. Overall, existing solutions can be divided into three categories:

*Work done during internship at Didichuxing Co. Ltd.
†Corresponding author.

Preprint. Under review.

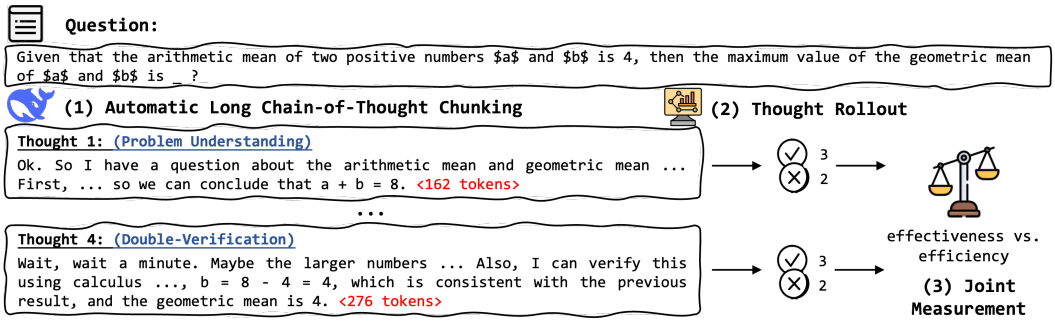

Figure 1: The workflow of how to investigate the importance of thoughts within a long CoT.

training-free, *supervised fine-tuning (SFT) based*, and *reinforcement learning (RL) based*. Training-free methods prompt LLMs to use limit word count to answer given question [11, 12], or explicitly set the maximum token-budget [13] in inference stage, e.g., Qwen3[14]. SFT-based approaches often rely on rejection sampling [15], which selects correct but shortest reasoning trajectories through repetitive sampling. For example, UPFT [16] and C3oT [17] directly use selected trajectories to fine-tune LLMs for reasoning token length compression [18, 19, 20]. RL-based methods employ offline or online learning strategy [21] for CoT-length preference learning. On the one hand, offline learning methods like DAST [22] and O1-Pruner [19] construct offline pairwise samples by treating the correct shorter response as positive and the correct longer response as negative, and applies RL algorithms [23] to optimize length preferences [24]. On the other hand, online RL methods such as Kimi k1.5 [25] and DAPO [26] incorporate length-aware reward functions to penalize unnecessary reasoning steps. However, these efforts **equally compress all thoughts within a long CoT** without considering differences in their importance, e.g., as shown in Figure 1, "double-verification" thought might be less important than the "problem understanding" as it doesn't bring extra accuracy but costs more tokens, which hinders more concise and effective reasoning.

To this end, we first investigate the importance of different thoughts within a long CoT by examining their contributions to reasoning. Specifically, we use a three-step approach shown in Figure 1: *(1) Automatic Long CoT Chunking:* Unlike existing work that relies thought templates [27, 28, 29] or split characters (e.g., '\n\n' [18] or 'wait' [30]), we propose to automatically split long CoT distilled from recent advanced LLMs (e.g., DeepSeek-R1) into multiple thought chunks, facilitating thought-level analysis. *(2) Thought Rollout:* Different from previous works [20] that use token probability to quantify, we run Monte Carlo rollouts [2] of each thought to approximate their effectiveness and efficiency contribution for the reasoning process. We find that front thoughts tend to yield a higher contribution, even enabling Qwen2.5-7B to achieve a performance comparable to its distillation version, DeepSeek-R1-Distill-Qwen-7B, under a zero-shot prompting strategy. *(3) Joint Measurement:* Building upon the insights, we propose a joint metric that simultaneously measures both the effectiveness and efficiency contribution of each thought. Our theoretical analysis proves that the deviation of the proposed metric from the optimal one is upper-bounded.

Based on the above thought measurement approach, we propose **Long⊗Short**, an efficient reasoning framework that enables two LLMs to collaboratively solve the problem: a long-thought LLM to generate important thoughts, while a short-thought LLM for remaining thoughts, as shown in Table 1. Specifically, we first fine-tune LLMs to follow long-thought reasoning (i.e., <think>...</think>) and short-thought reasoning (i.e., <answer>...</answer> or <answer>...</rethink>) styles under the supervision of synthesized cold start instructions. Moreover, we propose a *Synergizing-oriented Multi-Turn Reinforcement Learning* scheme to further enhance the efficient collaborative reasoning capability of the proposed framework. Different from existing studies [31] that rely on interactions with the environment to obtain reward signals [32, 33], our long-thought and short-thought LLMs engage in a multi-turn conversation to arrive at a final answer reward. Based on asynchronous policy optimization, the collaboration between long-thought and short-thought LLM could become stable, and their own reasoning capability could be improved, naturally emerging the oha moment [5, 34] of "Overthinking" (e.g., maybe i'm overthinking this. perhaps there's a simpler way) and "Rethinking" (e.g., current approach need to be reconsidered), as illustrated in Table 1.

We validate Long⊗Short on five widely used benchmarks, including MATH 500, AIME 2024, AIME 2025, AMC 2023, and GPQA Diamond. The experimental results demonstrate that Long⊗Short

Table 1: An illustrative example how a long-thought LLM and a short-thought LLM switch role to answer a given question. In general, long-thought LLM uses <think> to start its thought, and use </think> to stop thinking process. The short-thought LLM use <answer> to continue remaining steps, while use </answer> to stop solving process or use </rethink> to request the long-thought LLM to generate thoughts again.

Question: Given that the arithmetic mean of two positive numbers a and b is 4, then the maximum value of the geometric mean of a and b is ?
Ground Truth: 4
Long⊗Short:
<think> Ok. So I have a question ... </think>
<answer> Let's ... Therefore, current approach need to be reconsidered. </rethink>
...
<think> Wait, ... wait, hold on. maybe i'm overthinking this. perhaps there's a simpler way. </think>
<answer> Therefore, the answer should be 4. </answer>

enables current LLM backbones (e.g., Qwen2.5-7B and Llama3.1-8B) to achieve comparable performance compared to their distillation versions (e.g., DeepSeek-R1-Distill-Qwen-7B and DeepSeek-R1-Distill-Llama-8B), while reducing average token length by over 80%.

In summary, our key contributions are threefold: (1) We establish a thought analysis framework, which automatically chunks long CoT into multiple thoughts and quantifies their contributions. (2) We propose Long⊗Short, which adopts cold-start and multi-turn RL training to synergize long-thought and short-thought reasoning. (3) Extensive experiments and theoretical justification demonstrate the effectiveness of our method, offering a foundation for long CoT compression.

2 Preliminaries

Definition 1. ***Thought.*** *A thought is the basic logical block (e.g., problem decomposition, solution verification) in a LLM reasoning process. Given a long CoT response y, it can be structured as an ordered sequence of thoughts: $y = \{y_1, y_2, \ldots, y_n\}$, where n is the total number of thought.*

Building on this, we further propose the long⊗short thought reasoning paradigm that strategically switches between two LLMs to generate thoughts:

Problem 1. ***Long⊗Short Thought Reasoning.*** *This work assume that a long CoT reasoning process $y = \{y_1, y_2, \ldots, y_n\}$ can be reformulated into a mixture of long and short thoughts, i.e., $y = \{l_1, s_1, l_2, s_2, \ldots, l_m, s_k\}$, where l_i represents a important long thought and s_i denotes a corresponding compressed short thought. The inequality $m + k \leq n$ holds, as multiple thoughts might be compressed into a single short thought.*

This reasoning paradigm enables reasoning process to dynamically alternate between long-thought and short-thought reasoning. As shown in Table 1, the long-thought and short-thought LLMs collaboratively switch roles in order to solve the given question.

3 Understanding Thoughts within Long CoT

To achieve the aforementioned long⊗short thought reasoning, we first investigate thought importance, i.e., how different thoughts influences model effectiveness and efficiency.

3.1 Quantitative Analysis by Automatic Long CoT Chunking and Thought Rollout

Our idea is straightforward: given a question q and the long CoT response y from the reasoning model π_θ, we first prompt LLMs to split the long CoT into multiple thought chunks $\{y_1, y_2, \ldots, y_n\}$ shown in Figure 1, each a logical reasoning block (e.g., problem understanding and answer verification). Prompt template is in Appendix A.2.2. Then, we investigate the effectiveness and efficiency contribution of each thought by running Monte Carlo rollouts [2] on its base model $\pi_{\theta_{base}}$:

Assumption 1. *Given a question q and the split thought chunks $y = \{y_1, y_2, \ldots, y_n\}$ from reasoning model π_θ, a thought y_i is important if it can help original base model generate a response $y_{base} = \pi_{\theta_{base}}(q, \{y_1, y_2, \ldots, y_i\})$, which achieves higher accuracy with small response length increase.*

With this assumption, we perform a detailed quantitative analysis to investigate how different thoughts influence model effectiveness and efficiency. Specifically, we select DeepSeek-R1-Distill-Qwen-7B [5] as our long CoT model, which may reflect the accuracy upper bound with all long thought, while using Qwen2.5-7B to indicate the base accuracy and length under the occasion where all thoughts are compressed into short thoughts. We rollout at each thought and report the Pass@1 accuracy and response length. Figure 2 shows on the results on MATH 500 and GPQA Diamond. The following key findings were observed: **(1) The front thoughts bring more accuracy gain:** As shown in Figure 2(a), the accuracy of base model consistently increase when using top-0 to top-10 thoughts to rollout. This trend is consistent across all datasets, indicating that thoughts positioned earlier are more critical for effectiveness. Notably, in the GPQA Diamond dataset, we observe that the front thoughts (about 8th thoughts) even enable the Qwen2.5-7B to largely outperform DeepSeek-R1-Distill-Qwen-7B on such a totally training-free rollout strategy. These kind of thoughts intuitively should be important. **(2) Long thoughts lead to significant length increase:** As shown in Figure 2(b), the length of base model significantly increase when using thought to rollout. On GPQA Diamond dataset, even only using top-1 thought for base model doubles the response length (from roughly 2.5×10^3 to 5×10^3). This reminds us to consider the efficiency of thought, for thought that significantly increase response length to achieve only slight accuracy improvement, we may consider to compress these thoughts.

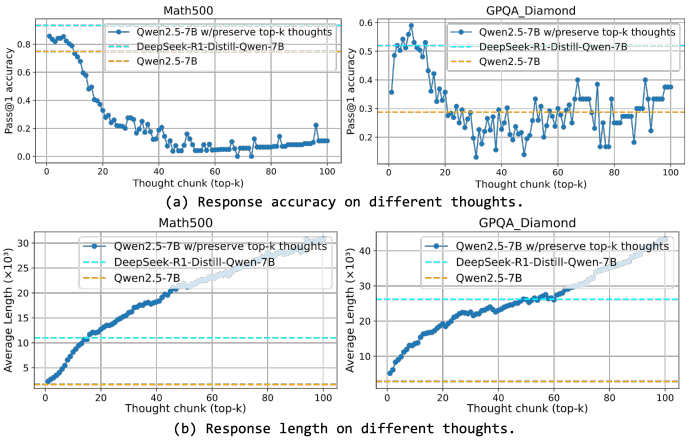

Figure 2: Quantitative illustration how thoughts affect (a) response accuracy and (b) response length on two reasoning dataset. We investigate how can long thoughts distilled from DeepSeek-R1-Distill-Qwen-7B affect the accuracy and length of base model Qwen2.5-7B. The cyan dashed line can be viewed as the effectiveness upper bound with full long thoughts, whereas the orange dashed line indicates that under full short thoughts.

These experimental observations demonstrate that both the accuracy gains and the increased length caused by thoughts are important. However, it still a non-trivial problem to justify thought importance based on these scattered analytical evidence. A unified framework considering thought effectiveness and efficiency should be proposed.

3.2 Joint Measurement of Thought Effectiveness and Efficiency

In this section, we propose a unified metric to jointly evaluate the effectiveness and efficiency of each thought. This metric is formulated by considering the accuracy gain from Monte Carlo rollouts alongside factors penalizing excessive length. The metric for thought y_i is defined as:

$$\mathcal{M}(y_i) = \log_2 \left(1 + \left(\frac{d_y - d_{\{y_1, y_2, \ldots, y_i\}}}{d_y} \right) \cdot \left(\frac{d_y - d_{y_i}}{d_y} \right) \cdot \left(\frac{N_i^{right}}{N_i^{sum}} \right) \right) - \delta(y_i), \qquad (1)$$

where (1) d_y is the sum of length of all thoughts; (2) d_{y_i} is the length of the specific thought y_i. This term $\frac{d_y - d_{y_i}}{d_y}$ assigns higher scores to shorter thoughts; (3) $d_{\{y_1,\ldots,y_i\}}$ is the cumulative length of thoughts from y_1 up to y_i. The term $\frac{d_y - d_{\{y_1,\ldots,y_i\}}}{d_y}$ thus assigns higher scores to thoughts with shorter context length; (4) $\frac{N_i^{\text{right}}}{N_i^{\text{sum}}}$ represents the empirical accuracy obtained by executing Monte Carlo rollouts process $\pi_{\theta_{base}}(q, \{y_1, y_2, \ldots, y_i\})$. N_i^{sum} is the total number of rollouts, and N_i^{right} is the count of correct responses. The higher this term, the greater the effectiveness of thought; (5) $\delta(y_i) \in (0, 1)$ is a conditional decay penalty (set to 0.25 in our experiments). It is activated if the inclusion of thought y_i does not lead to accuracy gain $\frac{N_i^{\text{right}}}{N_i^{\text{sum}}} \leq \frac{N_{i-1}^{\text{right}}}{N_{i-1}^{\text{sum}}}$ compared to using thoughts y_{i-1}.

This term assigns lower score to redundant thoughts, which cannot bring extra accuracy gain.

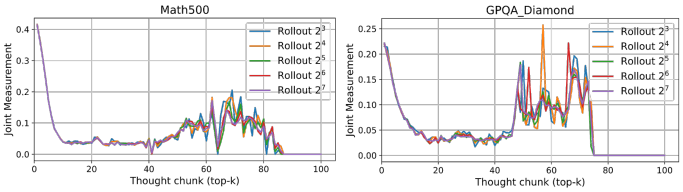

Figure 3: Joint measurement of thought effectiveness and efficiency when Monte Carlo rollouts times increasing from 2^3 to 2^7.

We present the measurement results in Figure 3, where the total times of rollout increasing from 2^3 to 2^7. The results align with our intuition and analysis: high scores are predominantly associated with front thoughts, while only a few rear thoughts occasionally trigger a resurgence in peak values. In addition, as the number of Monte Carlo rollouts increases, the proposed approximate metrics gradually converges to a theoretical curve. To provide more comprehensive understanding, we further analyze the error bound of our proposed measurement. We report the resources cost in Appendix A.3.2.

3.3 Theoretical Justification

Based on the structure of Equation 1, we assume that the optimal measurement accounts for the true probability of success given the thoughts up to y_i, P_i^{true}. Let the length-based factors $T_1(i) = \frac{d_y - d_{\{y_1,\ldots,y_i\}}}{d_y}$ and $T_2(i) = \frac{d_y - d_{y_i}}{d_y}$, we can write the optimal measurement as:

$$\mathcal{M}_{opt}(y_i) = \log_2\left(1 + T_1(i) \cdot T_2(i) \cdot P_i^{true}\right) - \delta_(y_i). \tag{2}$$

The error $\mathcal{E}(y_i)$ of our proposed measurement $\mathcal{M}(y_i)$ relative to the theoretical optimum $\mathcal{M}_{opt}(y_i)$ is given by their difference:

$$\mathcal{E}(y_i) = \mathcal{M}(y_i) - \mathcal{M}_{opt}(y_i) = \log_2\left(\frac{1 + T_1(i)T_2(i)\hat{P}_i}{1 + T_1(i)T_2(i)P_i^{true}}\right), \tag{3}$$

where $\hat{P}_i = \frac{N_i^{right}}{N_i^{sum}}$ is the Monte Carlo estimate of P_i^{true}. To bound the absolute error $|\mathcal{E}(y_i)|$, we note that it is determined by the term $|\log_2(A) - \log_2(B)|$, where $A = 1 + T_1(i)T_2(i)\hat{P}_i$ and $B = 1 + T_1(i)T_2(i)P_i^{true}$. Using the Mean Value Applying the Mean Value Theorem [35], there exists ξ between A and B such that $\log_2(A) - \log_2(B) = (A - B)\frac{1}{\xi \ln 2}$. Thus:

$$|\mathcal{E}(y_i)| \leq |\log_2(A) - \log_2(B)| = \frac{T_1(i)T_2(i)|\hat{P}_i - P_i^{true}|}{|\xi| \ln 2}. \tag{4}$$

Since thoughts have non-zero length ($d_{y_i} > 0$) and the sequence does not cover the full length before thought n ($d_{\{y_1,\ldots,y_i\}} < d_y$ for $i < n$), we have $T_1(i) \in [0, 1)$ and $T_2(i) \in [0, 1)$. Also, $\hat{P}_i, P_i^{true} \in [0, 1]$. Therefore, $A, B \in [1, 2]$, which implies $\xi \in [1, 2]$ and $|\xi| \geq 1$. $|\hat{P}_i - P_i^{true}| \leq \epsilon$ where ϵ decreases with N_i^{sum}. Combining these results, we obtain the following upper bound:

$$|\mathcal{E}(y_i)| \leq \frac{\epsilon}{\ln 2}. \tag{5}$$

This bound reveals that the error is mainly due to the probability error ϵ of Monte Carlo rollout, and can be reduced by increasing the sample size. The detailed derivation is in the Appendix A.1.

4 Long⊗Short

Building on these analysis, we propose Long⊗Short, a efficient reasoning framework to incentivize the long⊗short thought reasoning capacity in LLMs. Figure 4 illustrates its overall pipeline.

4.1 Long⊗Short Thought Reasoning Cold Start with Supervised Fine-Tuning

With the proposed automatic long CoT chunking and joint measurement method, we can obtain a series of pairs containing thoughts and their scores, i.e., $\{y, \mathcal{M}(y)\} = \{(y_1, \mathcal{M}(y_1)), (y_2, \mathcal{M}(y_2)), \ldots, (y_n, \mathcal{M}(y_n))\}$, where $\mathcal{M}(.)$ is the proposed joint measurement

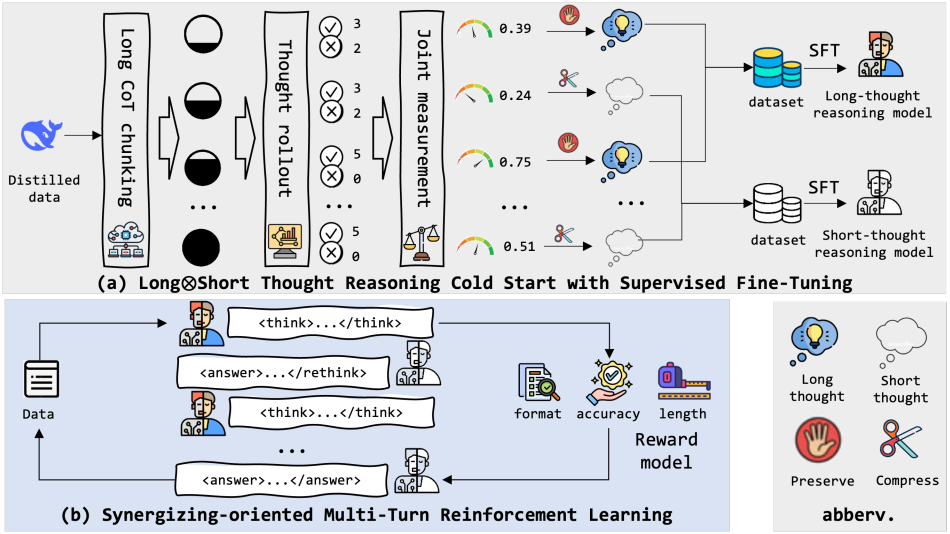

Figure 4: An overview of Long⊗Short framework.

of thought effectiveness and efficiency. As shown in Figure 4, the next step is to construct SFT dataset for fine-tuning long-thought and short-thought LLMs.

Cold Start Data Synthesization. In this paper, thoughts with high scores are assigned to the long-thought LLM, and those with lower scores are handled by the short-thought LLM. To achieve this, we adopt a heuristic approach via a sequential scan over the trajectory of thoughts $\{(y_1, \mathcal{M}(y_1)), (y_2, \mathcal{M}(y_2)), \ldots, (y_n, \mathcal{M}(y_n))\}$. In this procedure, the first thought, y_1, is always preserved as a long thought. For each subsequent thought y_i (for $i > 1$)), we compare its joint measurement $\mathcal{M}(y_i)$ against the maximum score of all previous thoughts: $\mathcal{M}(y_i) > \max\{\mathcal{M}(y_1), \mathcal{M}(y_2), \ldots, \mathcal{M}(y_{i-1})\}$. If this inequality holds, y_i is deemed sufficiently important and is preserved as a long thought. Otherwise, if $\mathcal{M}(y_i) \leq \max\{\mathcal{M}(y_1), \ldots, \mathcal{M}(y_{i-1})\}$, a non-reasoning LLM is prompted to re-complete y_i based on the thought type, and the resulted output naturally serves as a short thought. This procedure ultimately produces a structured sequence comprising alternating thought chunks: $\{l_1, s_1, l_2, s_2, \ldots, l_m, s_k\}$, where $m + k \leq n$ since multiple thoughts may merge into a single short thought. The prompt template, chunked thought statistics, rollout cost and SFT dataset statistics is in Appendix A.2 and A.3.1.

Efficiency-Aware Fine-Tuning. Subsequently, we concatenate reasoning template and thought context to obtain two SFT datasets D_{long} and D_{short}. Specifically, for D_{long}, the instruction input $x = \Gamma_{long} \parallel q \parallel h$ is the concatenation of the long-thought reasoning template Γ_{long}, question q, and historical conversation h (e.g., <think>...</think><answer>...</rethink>) between the long-thought and short-thought LLM. While the instruction output corresponds long-thought reasoning process. For D_{short}, the instruction input $x = \Gamma_{short} \parallel q \parallel h$. The difference lies that reasoning template and potential historical conversation h (e.g., <think>...</think> or <think>...</think><answer>...</rethink><think>...</think>). Detailed reasoning template and instruction example can be found in Appendix. Then, we utilize these datasets to perform full parameter fine-tuning [36] on a base model:

$$\mathcal{L}_{long/short} = -\mathbb{E}_{(x,o)\sim D_{long}/D_{short}} \left[\log \mathcal{P}_{\pi_{\theta_{long}}/\pi_{\theta_{short}}}(o \mid x) \right], \tag{6}$$

where x and o represent the instruction input and output in the reasoning trajectory, respectively. $\mathcal{L}_{long}$ and $\mathcal{L}_{short}$ are the loss of the long-thought LLM and the short-thought LLM. Through this fine-tuning process, we derive two specialized models: $\pi_{\theta_{long}}$, tailored for effectively generate long thought, and $\pi_{\theta_{short}}$, adapted for efficiently producing the short thought.

4.2 Synergizing-oriented Multi-Turn Reinforcement Learning

After the cold-start stage, the long-thought LLM and short-thought LLM can switch roles to collaboratively solve a question through multi-turn conversations. To facilitate more efficient reasoning, we

propose a synergizing-oriented multi-turn reinforcement learning framework [31, 32], designed to amplify the effectiveness and efficiency benefits of the long⊗short thought reasoning paradigm.

Asynchronous Policy Optimization. Unlike existing multi-turn RL methods [33, 32] that update the model's policy through direct interaction with the environment, our approach requires the long-thought LLM $\pi_{\theta_{long}}$ and the short-thought LLM $\pi_{\theta_{short}}$ to engage in a multi-turn conversation to derive the final reward for policy update. For the $\pi_{\theta_{long}}$, we formulate the optimization goal with a external short-thought LLM $\pi_{\theta_{short}}$ as follows:

$$\max_{\pi_{\theta_{long}}} \mathbb{E}_{x\sim\mathcal{D},o\sim\pi_{\theta_{long}}(\cdot|x;\pi_{\theta_{short}})}\left[r(x,o)\right] - \beta\mathbb{D}_{\mathrm{KL}}\left[\pi_{\theta_{long}}(y \mid x;\pi_{\theta_{short}})\|\pi_{\mathrm{ref}_{long}}(y \mid x;\pi_{\theta_{short}})\right], \tag{7}$$

where $x \sim \mathcal{D}$ represents the input, o is the output generated by $\pi_{\theta_{long}}$ given x and the external $\pi_{\theta_{short}}$, $r(x, o)$ is the reward function, $\pi_{\mathrm{ref}_{long}}$ is the reference LLM for the long-thought LLM, and $\mathbb{D}_{\mathrm{KL}}$ denotes the KL-divergence. By explicitly conditioning on an external model during policy learning, it enables more effective collaboration. Symmetrically, when optimizing the short-thought LLM $\pi_{\theta_{short}}$, the $\pi_{\theta_{long}}$ acts as the fixed external LLM:

$$\max_{\pi_{\theta_{short}}} \mathbb{E}_{x\sim\mathcal{D},o\sim\pi_{\theta_{short}}(\cdot|x;\pi_{\theta_{long}})}\left[r(x,o)\right] - \beta\mathbb{D}_{\mathrm{KL}}\left[\pi_{\theta_{short}}(y \mid x;\pi_{\theta_{long}})\|\pi_{\mathrm{ref}_{short}}(y \mid x;\pi_{\theta_{long}})\right], \tag{8}$$

where $\pi_{\mathrm{ref}_{short}}$ is the reference LLM. This alternating optimization ensures that each model update its own policy in synergy with its counterpart, enabling more efficient reasoning.

Online Sampling Strategy. To improve policy optimization stability and avoid the need for an additional value function approximation [37], we use Group Relative Policy Optimization (GRPO) [38] to sample a group of outputs $o = \{o_1, o_2, ..., o_G\}$ for the reference model for each input x. The sampling process follows an iterative framework where the system alternates between generating long thoughts (via $\pi_{\theta_{long}}$) and short thoughts (via $\pi_{\theta_{short}}$). This process continues until the short-thought LLM generates a final response, which is enclosed between designated answer tokens, `<answer>` and `</answer>`. Then, the reward is computed based on hybrid reward model.

Hybrid Reward Modeling. We now introduce the reward function $r(x, o)$, which is the primary training signal to guide the optimization process in reinforcement learning. We design a hybrid reward combining correctness, format following, and response length, formulated as:

$$r(x, o) = \eta \cdot \mathrm{EM}\left(a_{\text{pred}}, a_{\text{gold}}\right) + \lambda \cdot \mathrm{FM}\left(o\right) + \mu \cdot \mathrm{LM}\left(o\right), \tag{9}$$

where a_{pred} is the extracted final answer from output o and a_{gold} is the ground truth answer, the $\mathrm{EM}(.)$ is a rule-based function. $\mathrm{FM}\left(.\right)$ represents the format reward function, designed to encourage LLMs to follow the predefined training templates specified in Table 7 and 8 in Appendix A.2.1. We use the length reward function $\mathrm{LM}\left(.\right)$ reported in KIMI K1.5 [25] to guide model use fewer tokens to achieve the final answer. The coefficients η, λ, and μ act as reweighting parameters, enabling control over the contribution of each component in the large-scale reinforcement learning process. We set $\mu = 0$ in the initial stage and gradually increase it.

5 Experiments

5.1 Experimental Setup

Backbone LLMs. For our experiments, we selected **Llama-3.1-8B** [39] and **Qwen-2.5-7B** [40] as our base model, and compare with their long CoT reasoning version, i.e., **DeepSeek-R1-Distill-Llama-8B** and **DeepSeek-R1-Distill-Qwen-7B** [5].

Benchmarks. We evaluated the performance of our method on five widely used benchmarks: **MATH 500** [6], **AIME 2024** and **AIME 2025** [41], **GPQA Diamond** [8] and **AMC 2023** [42].

Evaluation and Metrics. We employ the Pass@1 accuracy, average length and Accuracy-Efficiency Score (AES) [19] as our metrics to assess whether the model achieves a desirable balance between reasoning effectiveness and efficiency. Specifically, $AES = \eta\frac{Length_{base}-Length_{model}}{Length_{base}} + \varsigma\left|\frac{Acc_{model}-Acc_{base}}{Acc_{base}}\right|$, where $Length_{base}$ and Acc_{base} refer to the response token length and accuracy of the distilled long CoT LLMs, respectively. Following original setting in O1-Pruner [19], we set $\eta = 1$ and $\varsigma = 3$ when $\frac{Acc_{model}-Acc_{base}}{Acc_{base}} \geq 0$, and $\varsigma = -5$ otherwise.

Table 2: Comparison of Long⊗Short in asynchronous evolution rounds shows a continual improvement in Pass@1 accuracy and a significant decrease in response length. We denote our model after the cold-start SFT stage as Long-r0⊗Short-r0, where the index indicates the round of policy evolution.

Round#	MATH 500 Pass@1 ↑	AMIE 2024 Pass@1 ↑	AMIE 2025 Pass@1 ↑	GPQA Diamond Pass@1 ↑	AMC 2023 Pass@1 ↑	Avg. Length ↓	Avg. AES ↑
DeepSeek-R1-Distill-Qwen-7B	**93.40**	**53.33**	**36.67**	**48.98**	**95.00**	24,566	0
Base (Qwen2.5-7B)	74.80	16.67	7.25	37.88	42.50	1,623	-1.33
Long-r0⊗Short-r0 w/SFT	80.40	26.67	13.33	44.44	67.50	7,323	-0.75
Long-r1⊗Short-r0	84.60	36.67	23.33	45.45	72.50	2,169	-0.08
Long-r2⊗Short-r0	87.60	43.33	26.67	46.46	85.00	2,331	0.32
Long-r2⊗Short-r1	87.00	33.33	26.67	45.96	82.50	**2,021**	0.12
Long-r3⊗Short-r1	88.60	43.33	30.00	46.96	87.50	2,457	0.43
Long-r3⊗Short-r2	87.80	43.33	26.67	46.96	87.50	2,116	0.38
Long-r4⊗Short-r2	89.80	46.67	33.33	47.97	90.00	2,113	**0.61**
DeepSeek-R1-Distill-Llama-8B	**87.20**	**43.33**	30.00	**49.49**	**90.00**	28,496	0
Base (Llama3.1-8B)	62.80	10.00	6.67	35.86	22.50	3,713	-1.83
Long-r0⊗Short-r0 w/SFT	73.40	23.33	16.67	40.40	47.50	6,611	-0.88
Long-r1⊗Short-r0	77.80	30.00	20.00	41.91	62.50	2,726	-0.23
Long-r2⊗Short-r0	80.20	36.67	20.00	42.93	72.50	2,925	0.10
Long-r2⊗Short-r1	79.40	33.33	16.67	41.91	67.50	2,406	-0.10
Long-r3⊗Short-r1	84.40	40.00	26.67	43.94	82.50	2,576	0.53
Long-r3⊗Short-r2	83.80	36.67	30.00	44.44	85.00	**2,376**	0.58
Long-r4⊗Short-r2	86.20	40.00	**33.33**	46.96	87.50	2,402	**0.81**

Implementation Details. We sample 0.1% of the OpenMathInstruct [43] dataset—1.8K mathematical reasoning problems with ground-truth answer—to distill long CoT responses from DeepSeek-R1, and then prompt the Qwen2.5-72B-Instruct model to perform automatic long CoT chunking [3]. By running Monte Carlo rollouts (5 times per thought) on a small LLM Qwen2.5-7B-Instruct and using Qwen2.5-72B-Instruct to complete short thought, we obtain an SFT dataset (1.4K samples total) for long-thought and short-thought reasoning model. For large-scale multi-turn RL training process, we utilize MATH-ligtheval [44] (7.5K training samples and 5K validation samples in total) to conduct iterative model self-evolution training. Details could be founded in Appendix A.3.

5.2 Main Results

We compare Long⊗Short across the SFT cold-start stage and iterative RL training process. As reported in Table 2. We highlight two key observations: **(1) Cold-start stage establishes a strong initial policy.** To enable the long-thought and short-thought LLMs to switch roles effectively, we perform Monte Carlo rollouts to collect SFT data. After fine-tuning, Qwen2.5-7B achieves significant performance gains: 7.4%, 59.98%, 83.86%, 17.32%, and 58.82% on MATH500, AIME24/25, GPQA Diamond, and AMC23, respectively, with an increase in average response length from 1,623 to 7,323 tokens. Similarly, Llama3.1-8B shows improvements of 16.88%, 133.30%, 149.93%, 12.67%, and 111.11%, with response length increasing from 3,713 to 6,611. The response length increase is expected as the long-thought LLM distills the long CoT reasoning style from DeepSeek-R1. The excessive reasoning length problem will be alleviated by subsequent RL process. **(2) Multi-turn RL continuously improve reasoning capacity.** As reported, after 4 rounds of evolution in the long-thought LLM and 2 rounds in the short-thought LLM, Long⊗Short enables Qwen2.5-7B and Llama3.1-8B to match the performance of DeepSeek-R1-Distill-Qwen-7B and DeepSeek-R1-Distill-Llama-8B, while reducing reasoning token lengths by over 80%. Long-r3⊗Short-r2 also achieves the best AES metric [45], confirming that iterative RL training maximizes the efficiency.

5.3 Comparison with Existing Baselines

We also present a comprehensive comparison between our proposed method and existing chain-of-thought long-to-short approaches. Specifically, we categorize the baseline approaches into the following groups: (1) **Training-free methods**: These rely on prompt engineering or inference-time techniques without any parameter updates. We select CoD [11] and TALE-EP [13] as our baselines. (2) **SFT-based methods**: These utilize supervised fine-tuning on curated datasets to learn reasoning patterns. We select C3oT [17], DAST [22] and O1-Pruner [19] as our baselines. (3)

[3]We release the chunked thought at https://huggingface.co/datasets/yasNing/OpenMath-ThoughtChunk1.8K.

Table 3: Comparison of Long⊗Short with existing approaches on Qwen2.5-7B. We selected the average accuracy and length of DeepSeek-R1-Distill-Qwen7B as our baseline for AES metric calculation.

Methods		MATH 500 Pass@1 ↑	AMIE 2024 Pass@1 ↑	AMIE 2025 Pass@1 ↑	GPQA Diamond Pass@1 ↑	AMC 2023 Pass@1 ↑	Avg. Length ↓	Avg. AES ↑
DeepSeek-R1-Distill-Qwen-7B		**93.40**	**53.33**	**36.67**	**48.98**	**95.00**	24,566	0
Base (Qwen2.5-7B)		74.80	16.67	7.25	37.88	42.50	1,623	-1.33
Training-free	CoD	82.60	46.67	23.33	35.86	75.00	13,229	-0.51
	TALE-EP	88.20	50.00	**36.67**	41.41	85.00	16,668	-0.08
SFT-based	C3oT	81.00	30.00	26.67	47.47	82.50	12,576	-0.42
	DAST	83.50	33.33	13.33	32.83	77.50	10,235	-0.74
	O1-Pruner	87.55	36.67	26.67	37.87	85.00	13,467	-0.37
RL-based	SimplePO-DAST	82.75	33.33	26.67	43.93	77.50	17,849	-0.69
	Kimi k1.5	84.00	43.33	26.67	37.37	75.00	17,583	- 0.65
Long⊗Short-Qwen2.5-7B		89.80	46.67	33.33	47.97	90.00	**2,113**	**0.61**

Table 4: Comparison of Long⊗Short with existing approaches on Llama3.1-8B. We selected the average accuracy and length of DeepSeek-R1-Distill-Llama8B as our baseline for AES metric calculation.

Methods		MATH 500 Pass@1 ↑	AMIE 2024 Pass@1 ↑	AMIE 2025 Pass@1 ↑	GPQA Diamond Pass@1 ↑	AMC 2023 Pass@1 ↑	Avg. Length ↓	Avg. AES ↑
DeepSeek-R1-Distill-Llama-8B		**87.20**	**43.33**	30.00	**49.49**	**90.00**	28,496	0
Base (Llama3.1-8B)		62.80	10.00	6.67	35.86	22.50	3,713	-1.83
Training-free	CoD	85.00	43.33	23.33	48.99	87.50	13,897	0.40
	TALE-EP	79.20	40.00	30.00	41.11	82.50	18,830	-0.11
SFT-based	C3oT	40.20	30.00	3.33	41.41	22.50	10,014	-2.06
	DAST	78.50	26.67	13.33	40.40	78.50	13,443	-0.52
	O1-Pruner	82.50	30.00	23.33	42.93	78.50	14,486	-0.22
RL-based	SimplePO-DAST	82.05	26.67	23.33	37.37	82.50	16,285	-0.37
	Kimi k1.5	76.50	13.33	3.33	30.30	72.50	13,684	-1.21
Long⊗Short-Llama3.1-8B		86.20	40.00	**33.33**	46.96	87.50	**2,402**	**0.81**

RL-based methods: These leverage reinforcement learning, typically with reward signals derived from task-specific objectives or human feedback, to optimize reasoning performance. We select SimplePO-DAST [22], and Kimi k1.5 [25] as our baselines.

As shown in Table 3 and Table 4, our method achieves consistent improvements in both reasoning accuracy and efficiency across multiple benchmarks and model backbones. **(1) On the Qwen2.5-7B backbone**, our method outperforms most baselines in terms of average accuracy while significantly reducing the average output length (2,113 tokens vs. 13K–17K for most baselines). Notably, our approach achieves a superior AES score, indicating that it not only maintains high reasoning performance but also generates more concise outputs compared to the AES baseline (DeepSeek-R1-Distill-Qwen7B). While some training-free and RL-based methods perform competitively on certain benchmarks, they often incur substantial output length and latency overheads. **(2) On the Llama3.1-8B backbone**, our method again demonstrates strong performance, achieving the highest accuracy on the AMIE 2025 benchmark (33.33%) and the second-best performance on MATH500 (86.20%), while maintaining the lowest average output length (2,402 tokens). Compared to the AES baseline (DeepSeek-R1-Distill-Llama8B), our method yields a notable AES improvement.

5.4 Comparison with Existing LLMs

In addition, we compare our proposed models, Long⊗Short-Qwen2.5-7B and Long⊗Short-Llama3.1-8B, with both reasoning and non-reasoning models to comprehensively evaluate their effectiveness and efficiency on reasoning tasks. We also compare our method with recently introduced hybrid-thinking-mode LLMs (e.g., Qwen3 [14]). Specifically, we categorize the baseline LLMs as follows: (1) **Non-reasoning models**: These models aim to solve problems quickly without engaging in explicit reasoning. While they offer high speed, the absence of step-by-step reasoning may result in incomplete

Table 5: Comparison with existing LLMs. We bold the results of models with comparable size to highlight the advantages of our efficient reasoning model over similarly sized LLMs (e.g. Qwen3-8B).

Methods	MATH 500 Pass@1 ↑	AMIE 2024 Pass@1 ↑	AMIE 2025 Pass@1 ↑	GPQA Diamond Pass@1 ↑	AMC 2023 Pass@1 ↑	Avg. Length ↓	Average Pass@1 ↑
Non-reasoning model							
Llama3.1-8B	62.80	10.00	6.67	35.86	22.50	3,713	27.50
Qwen2.5-7B	74.80	16.67	7.25	37.88	42.50	1,623	35.82
DeepSeek-V3-671B	90.20	39.20	30.00	69.10	90.00	3,287	63.70
GPT-4-o	78.60	20.00	13.33	48.48	50.00	2,840	42.08
Reasoning model							
DeepSeek-R1-Distill-Llama-8B	87.20	43.33	30.00	49.49	90.00	28,496	60.00
DeepSeek-R1-Distill-Qwen-7B	93.40	53.33	36.67	48.98	**95.00**	24,566	65.48
QwQ	94.40	46.67	60.00	46.67	85.00	19,504	66.55
DeepSeek-R1	**95.30**	**69.80**	**70.37**	71.50	**95.00**	26,874	80.39
OpenAI-o1	92.00	50.00	40.00	**72.73**	82.50	-	67.45
Hybrid-reasoning model							
Qwen3-8B-thinking	88.20	53.33	50.00	58.08	77.50	21,158	65.42
Qwen3-32B-thinking	94.40	66.67	53.33	63.64	92.50	18,607	74.11
Qwen3-8B-nothinking	81.60	36.67	20.00	53.03	60.00	5,580	50.26
Qwen3-32B-nothinking	85.40	36.67	20.00	61.62	77.50	4,301	56.24
Long⊗Short-Llama3.1-8B	86.20	40.00	33.33	46.96	87.50	2,402	58.80
Long⊗Short-Qwen2.5-7B	89.80	46.67	33.33	47.97	90.00	2,113	61.55

Table 6: Ablation study on Qwen2.5-7B.

Round#	MATH 500 Pass@1 ↑	AMIE 2024 Pass@1 ↑	GPQA Diamond Pass@1 ↑	Avg. Length ↓
Base (Qwen2.5-7B)	74.80	16.67	37.88	1,623
Prompt wo/SFT	71.60	6.67	33.83	5,362
SFT w/random	77.20	16.67	40.91	8,234
SFT w/ours	80.40	26.67	44.33	7,312

or less accurate outputs. We select advanced models such as GPT-4-o [4], DeepSeek-V3-671B [5], Qwen2.5-7B, and Llama3.1-8B as representative examples. (2) **Reasoning models**: These models employ extended chain-of-thought capabilities to perform multi-step reasoning across tasks. Although generally more effective, they tend to produce overly lengthy reasoning traces, which can affect efficiency. We include OpenAI-o1 [4], DeepSeek-R1 [5], QwQ [40], DeepSeek-R1-Distill-Qwen-7B, and DeepSeek-R1-Distill-Llama-8B as representative models. (3) **Hybrid-reasoning models**: These models are capable of switching between reasoning and non-reasoning modes, allowing for more flexible inference. We choose Qwen3-8B [14] and Qwen3-32B as our comparison.

As can be seen in Table 5, Long⊗Short demonstrates three key advantages: **(1) Compared with non-reasoning models**, Long⊗Short significantly improves performance across all benchmarks while maintaining similar response lengths. For instance, Long⊗Short-Llama3.1-8B achieves an average of over 20-point gain in most tasks compared to its base Llama3.1-8B with even shorter output length (2,402 vs. 3,713 tokens). **(2) Compared with reasoning models**, except for DeepSeek-R1, Long⊗Short achieves comparable or better performance while dramatically reducing token usage—compressing the average output length by more than 80% (e.g., from 24k–28k tokens down to 2k tokens), thus offering a more efficient solution for reasoning-intensive tasks. **(3) Compared with hybrid-reasoning models**, Long⊗Short consistently matches or surpasses their performance, while generating significantly shorter responses. For example, Long⊗Short-Qwen2.5-7B achieves 90.00 on AMC 2023, outperforming Qwen3-8B-nothinking (60.00) and closely matching Qwen3-32B-thinking (92.50), with less token length than Qwen3-8B-nothinking.

In general, our method achieves the highest average Pass@1 among all non-reasoning models of similar size, surpassing GPT-4-o, Qwen2.5-7B, Llama3.1-8B, Qwen3-8B-nothinking, and even Qwen3-32B-nothinking.

5.5 Ablation Study and In-Depth Analysis

We ablate the effectiveness of our two key component, i.e., long⊗short thought reasoning cold-start and synergizing-oriented multi-turn RL training, while also discuss the emerging of aha moment of long⊗short thought reasoning paradigm.

Ablation Study on Cold-Start. We evaluate the effectiveness of the proposed cold-start stage through the following variants: (1) *SFT w/random*, which replaces the thought metric with random

scoring; and (2) *Prompt wo/SFT*, which directly prompts the base model to adopt long-thought and short-thought reasoning styles without fine-tuning. As reported in Table 6, both of direct prompting or replacing thought metric with random value lead to a significant accuracy drop and length increase, highlighting the necessity of the cold-start stage for the long⊗short thought reasoning.

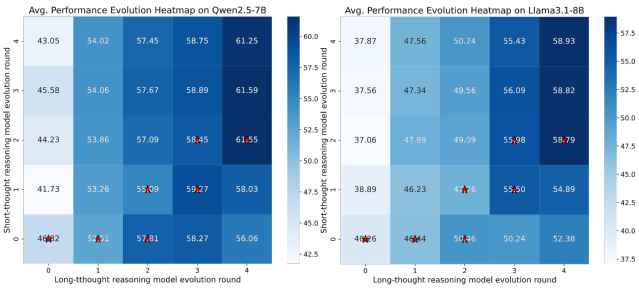

Figure 5: Performance evolution during asynchronous model policy update. We use red stars to indicate our model selection in the 4-round evolution process.

Model Evolution Analysis on Asynchronous Multi-Turn RL Training. During iterative RL, the long-thought and short-thought LLMs are updated asynchronously. Their policy changes influence each other's behavior and jointly affect the overall performance [46], as shown in Figure 5. In early training, updating the short-thought LLM degrades performance, highlighting the critical role of the long-thought LLM. To address this, we first evolve the long-thought LLM to round 2 while keeping the short-thought LLM fixed, achieving a significant performance gain. Subsequent updates to the short-thought LLM (to round 1 or beyond) initially cause performance drops, indicating instability in their collaboration. Performance stabilizes and improves as the long-thought LLM evolves further to rounds 3 and 4. While further gains are possible with continued evolution, we leave it for future work due to computing resource limitations.

Aha Moment of Long⊗Short Thought Reasoning. Another phenomenon is the aha moment of 'Overthinking' and 'Rethinking' emerged in RL training process. Although recent works [34, 47] reveal that even base model exhibit aha moment, we find that RL can cause the aha moment to occur frequently, accompanied by a reduction in the response length. We report more detailed analysis of aha moment in Appendix A.4.3.

6 Related Work

6.1 Reinforcement Learning for LLM Reasoning.

Recently, reinforcement learning (RL) [48, 49] has been widely adopted to improve LLM reasoning capability. For instance, Proximal Policy Optimization (PPO) [50] trains a reward model on human preferences and fine-tunes the LLM policy accordingly. While effective, PPO depends heavily on human-annotated preference data. To simplify training, methods like DPO [23] and SimPO [51] have been introduced for direct policy optimization without preference annotation requirements. More recently, GRPO [37, 38] further simplifies this pipeline by removing the critic model, instead estimating policy value based on a group of rollout responses. However, these single-turn RL methods cannot interact with environments [52], limiting multi-step LLM reasoning. To this end, the multi-turn RL [31] is proposed to improve LLM reasoning by enabling collaboration with external tools [53, 33] and LLMs [32]. Nevertheless, the application of multi-turn RL to facilitate efficient reasoning remains largely unexplored.

6.2 Efficient Chain-of-Thought Reasoning in LLMs.

Long chain-of-thought (CoT) reasoning emerged in recent LLMs (e.g., OpenAI-o1 [4] and DeepSeek-R1 [5]) has demonstrated significant improvement on complex reasoning tasks [6, 8]. To improve reasoning efficiency [9], recent work focuses on reducing CoT length without sacrificing accuracy [54], which can be mainly divided into three categories: (1) *Training-free* methods explicitly prompt LLMs to generate limited outputs [11, 12] or enforce hard token limits during inference [13, 55, 14]. They avoid parameter updates and rely on external constraints to guide brevity [56, 57, 58]. (2) *SFT-based* methods first apply rejection sampling to obtain correct but shorter reasoning trajectories. Then, approaches like TOPS [59] and C3oT [17] directly use such trajectories for SFT [18, 19, 20, 10, 60]. This strategy encourages LLM to conduct more concise reasoning [61, 62, 17, 63, 64, 65, 66]. (3) *RL-based* methods employ both offline and online learning strategy [21] for optimizing length-aware preferences. Offline methods like DAST [22] construct shorter-longer response pairs [63] for CoT-length preference learning, while online methods such as Kimi k1.5 [25] and DAPO [26] incorporate

length rewards to penalize excessive reasoning [24, 67, 45, 68]. However, these methods equally compress all thoughts within long CoT without considering thought importance difference, limiting more efficient reasoning.

7 Conclusion, Limitation and Future Work

In this work, we propose a theoretically grounded thought analysis framework that incorporates an automatic chunking method and a Monte Carlo thought rollout process to quantify thought importance. Moreover, we introduce Long⊗Short, a collaborative reasoning framework that sequentially execute SFT for reasoning style cold-start and multi-turn RL for model self-evolution, enabling two LLMs to dynamically switch between long-thought and short-thought reasoning styles. Extensive experiments show that Long⊗Short significantly enhances the performance of base LLMs while maintaining competitive efficiency. However, our method incurs substantial computational cost due to Monte Carlo rollouts and iterative multi-turn RL evolution. Future work includes scaling long⊗short thought reasoning with size-varying LLMs and applying it to potentially industrial applications.

References

[1] OpenAI. Learning to reason with llms. *https://openai.com/index/learnin g-to-reason-with-llms/*, 2024.

[2] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[4] OpenAI. Introducing openai o1. *https://openai.com/o1/*, 2024.

[5] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[6] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

[8] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.

[9] Mingyu Jin, Qinkai Yu, Dong Shu, Haiyan Zhao, Wenyue Hua, Yanda Meng, Yongfeng Zhang, and Mengnan Du. The impact of reasoning step length on large language models. *arXiv preprint arXiv:2401.04925*, 2024.

[10] Jeffrey Cheng and Benjamin Van Durme. Compressed chain of thought: Efficient reasoning through dense representations. *arXiv preprint arXiv:2412.13171*, 2024.

[11] Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025.

[12] Abhinav Kumar, Jaechul Roh, Ali Naseh, Marzena Karpinska, Mohit Iyyer, Amir Houmansadr, and Eugene Bagdasarian. Overthink: Slowdown attacks on reasoning llms. *arXiv e-prints*, pages arXiv–2502, 2025.

[13] Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-budget-aware llm reasoning. *arXiv preprint arXiv:2412.18547*, 2024.

[14] Qwen Team. Qwen3 technical report. *https://github.com/QwenLM/Qwen3*, 2025.

[15] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

[16] Ke Ji, Jiahao Xu, Tian Liang, Qiuzhi Liu, Zhiwei He, Xingyu Chen, Xiaoyuan Liu, Zhijie Wang, Junying Chen, Benyou Wang, et al. The first few tokens are all you need: An efficient and effective unsupervised prefix fine-tuning method for reasoning models. *arXiv preprint arXiv:2503.02875*, 2025.

[17] Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24312–24320, 2025.

[18] Jianyuan Zhong, Zeju Li, Zhijian Xu, Xiangyu Wen, and Qiang Xu. Dyve: Thinking fast and slow for dynamic process verification. *arXiv preprint arXiv:2502.11157*, 2025.

[19] Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*, 2025.

[20] Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*, 2025.

[21] Julian Schrittwieser, Thomas Hubert, Amol Mandhane, Mohammadamin Barekatain, Ioannis Antonoglou, and David Silver. Online and offline reinforcement learning by planning with a learned model. *Advances in Neural Information Processing Systems*, 34:27580–27591, 2021.

[22] Yi Shen, Jian Zhang, Jieyun Huang, Shuming Shi, Wenjing Zhang, Jiangze Yan, Ning Wang, Kai Wang, and Shiguo Lian. Dast: Difficulty-adaptive slow-thinking for large reasoning models. *arXiv preprint arXiv:2503.04472*, 2025.

[23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36:53728–53741, 2023.

[24] Daman Arora and Andrea Zanette. Training language models to reason efficiently. *arXiv preprint arXiv:2502.04463*, 2025.

[25] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.

[26] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[27] Ling Yang, Zhaochen Yu, Bin Cui, and Mengdi Wang. Reasonflux: Hierarchical llm reasoning via scaling thought templates. *arXiv preprint arXiv:2502.06772*, 2025.

[28] Ziyu Wan, Yunxiang Li, Yan Song, Hanjing Wang, Linyi Yang, Mark Schmidt, Jun Wang, Weinan Zhang, Shuyue Hu, and Ying Wen. Rema: Learning to meta-think for llms with multi-agent reinforcement learning. *arXiv preprint arXiv:2503.09501*, 2025.

[29] Simon A Aytes, Jinheon Baek, and Sung Ju Hwang. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*, 2025.

[30] Ximing Lu, Seungju Han, David Acuna, Hyunwoo Kim, Jaehun Jung, Shrimai Prabhumoye, Niklas Muennighoff, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, et al. Retro-search: Exploring untaken paths for deeper and efficient reasoning. *arXiv preprint arXiv:2504.04383*, 2025.

[31] Lior Shani, Aviv Rosenberg, Asaf Cassel, Oran Lang, Daniele Calandriello, Avital Zipori, Hila Noga, Orgad Keller, Bilal Piot, Idan Szpektor, et al. Multi-turn reinforcement learning from preference human feedback. *arXiv preprint arXiv:2405.14655*, 2024.

[32] Yifei Zhou, Song Jiang, Yuandong Tian, Jason Weston, Sergey Levine, Sainbayar Sukhbaatar, and Xian Li. Sweet-rl: Training multi-turn llm agents on collaborative reasoning tasks. *arXiv preprint arXiv:2503.15478*, 2025.

[33] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

[34] Shu Yang, Junchao Wu, Xin Chen, Yunze Xiao, Xinyi Yang, Derek F Wong, and Di Wang. Understanding aha moments: from external observations to internal mechanisms. *arXiv preprint arXiv:2504.02956*, 2025.

[35] Carl Ludwig Siegel. A mean value theorem in geometry of numbers. *Annals of Mathematics*, 46(2):340–347, 1945.

[36] Kai Lv, Yuqing Yang, Tengxiao Liu, Qinghui Gao, Qipeng Guo, and Xipeng Qiu. Full parameter fine-tuning for large language models with limited resources. *arXiv preprint arXiv:2306.09782*, 2023.

[37] Shyam Sundhar Ramesh, Yifan Hu, Iason Chaimalas, Viraj Mehta, Pier Giuseppe Sessa, Haitham Bou Ammar, and Ilija Bogunovic. Group robust preference optimization in reward-free rlhf. *Advances in Neural Information Processing Systems*, 37:37100–37137, 2024.

[38] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[39] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[40] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

[41] MAA Committees. Aime problems and solutions. *https://artofproblemsolving.com/wiki/ index.php*, 2025.

[42] MAA MAC Committees. American mathematics competitions. *https://huggingface.co/datasets/zwhe99/amc23*, 2023.

[43] Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. Openmathinstruct-1: A 1.8 million math instruction tuning dataset. *Advances in Neural Information Processing Systems*, 37:34737–34774, 2024.

[44] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[45] Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning. *arXiv preprint arXiv:2504.01296*, 2025.

[46] Xinyu Guan, Li Lyna Zhang, Yifei Liu, Ning Shang, Youran Sun, Yi Zhu, Fan Yang, and Mao Yang. rstar-math: Small llms can master math reasoning with self-evolved deep thinking. *arXiv preprint arXiv:2501.04519*, 2025.

[47] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[48] Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. Reinforcement learning: A survey. *Journal of artificial intelligence research*, 4:237–285, 1996.

[49] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[50] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[51] Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward. *Advances in Neural Information Processing Systems*, 37:124198–124235, 2024.

[52] Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, Jérémy Scheurer, Javier Rando, Rachel Freedman, Tomasz Korbak, David Lindner, Pedro Freire, et al. Open problems and fundamental limitations of reinforcement learning from human feedback. *arXiv preprint arXiv:2307.15217*, 2023.

[53] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.

[54] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Hanjie Chen, Xia Hu, et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025.

[55] Ayeong Lee, Ethan Che, and Tianyi Peng. How well do llms compress their own chain-of-thought? a token complexity approach. *arXiv preprint arXiv:2503.01141*, 2025.

[56] Zishun Yu, Tengyu Xu, Di Jin, Karthik Abinav Sankararaman, Yun He, Wenxuan Zhou, Zhouhao Zeng, Eryk Helenowski, Chen Zhu, Sinong Wang, et al. Think smarter not harder: Adaptive reasoning with inference aware optimization. *arXiv preprint arXiv:2501.17974*, 2025.

[57] Yuyang Wu, Yifei Wang, Tianqi Du, Stefanie Jegelka, and Yisen Wang. When more is less: Understanding chain-of-thought length in llms. *arXiv preprint arXiv:2502.07266*, 2025.

[58] Yuan Sui, Yufei He, Tri Cao, Simeng Han, and Bryan Hooi. Meta-reasoner: Dynamic guidance for optimized inference-time reasoning in large language models. *arXiv preprint arXiv:2502.19918*, 2025.

[59] Wenkai Yang, Shuming Ma, Yankai Lin, and Furu Wei. Towards thinking-optimal scaling of test-time compute for llm reasoning. *arXiv preprint arXiv:2502.18080*, 2025.

[60] Xinyin Ma, Guangnian Wan, Runpeng Yu, Gongfan Fang, and Xinchao Wang. Cot-valve: Length-compressible chain-of-thought tuning. *arXiv preprint arXiv:2502.09601*, 2025.

[61] Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. Codi: Compressing chain-of-thought into continuous space via self-distillation. *arXiv preprint arXiv:2502.21074*, 2025.

[62] Tergel Munkhbat, Namgyu Ho, Seo Hyun Kim, Yongjin Yang, Yujin Kim, and Se-Young Yun. Self-training elicits concise reasoning in large language models. *arXiv preprint arXiv:2502.20122*, 2025.

[63] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, et al. Do not think that much for 2+ 3=? on the overthinking of o1-like llms. *arXiv preprint arXiv:2412.21187*, 2024.

[64] Yingqian Cui, Pengfei He, Jingying Zeng, Hui Liu, Xianfeng Tang, Zhenwei Dai, Yan Han, Chen Luo, Jing Huang, Zhen Li, et al. Stepwise perplexity-guided refinement for efficient chain-of-thought reasoning in large language models. *arXiv preprint arXiv:2502.13260*, 2025.

[65] Jintian Zhang, Yuqi Zhu, Mengshu Sun, Yujie Luo, Shuofei Qiao, Lun Du, Da Zheng, Huajun Chen, and Ningyu Zhang. Lightthinker: Thinking step-by-step compression. *arXiv preprint arXiv:2502.15589*, 2025.

[66] Yuchen Yan, Yongliang Shen, Yang Liu, Jin Jiang, Mengdi Zhang, Jian Shao, and Yueting Zhuang. Inftythink: Breaking the length limits of long-context reasoning in large language models. *arXiv preprint arXiv:2503.06692*, 2025.

[67] Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.

[68] Mehdi Fatemi, Banafsheh Rafiee, Mingjie Tang, and Kartik Talamadupula. Concise reasoning via reinforcement learning. *arXiv preprint arXiv:2504.05185*, 2025.

A Appendix

A.1 Theoretical Justification

We first give the mean value theorem lemma [35] used for our theoretical justification:

Lemma 1 (Mean Value Theorem)**.** *Let $f : [a, b] \to \mathbb{R}$ be a continuous function on the closed interval $[a, b]$ and differentiable on the open interval (a, b). Then there exists a point $c \in (a, b)$ such that*

$$f'(c) = \frac{f(b) - f(a)}{b - a}. \tag{10}$$

With the defined lemma, we now analyze the error of our proposed measurement. we first postulate a form for the theoretical optimal measurement $\mathcal{M}_{opt}(y_i)$. Based on the structure of Equation 1, we assume that the optimal measurement accounts for the true probability of success given the thoughts up to y_i, P_i^{true}. Let the length-based factors $T_1(i) = \frac{d_y - d_{\{y_1,\ldots,y_i\}}}{d_y}$ and $T_2(i) = \frac{d_y - d_{y_i}}{d_y}$, we can write the optimal measurement as:

$$\mathcal{M}_{opt}(y_i) = \log_2 \left(1 + T_1(i) \cdot T_2(i) \cdot P_i^{true}\right) - \delta_(y_i). \tag{11}$$

Then, The error $\mathcal{E}(y_i)$ of our proposed measurement $\mathcal{M}(y_i)$ relative to the theoretical optimum $\mathcal{M}_{opt}(y_i)$ is given by their difference:

$$\begin{aligned}
\mathcal{E}(y_i) &= \mathcal{M}(y_i) - \mathcal{M}_{opt}(y_i) \\
&= \left[\log_2 \left(1 + T_1(i)T_2(i)\hat{P}_i\right) - \delta(y_i)\right] - \left[\log_2 \left(1 + T_1(i)T_2(i)P_i^{true}\right) - \delta_(y_i)\right] \\
&= \left[\log_2 \left(1 + T_1(i)T_2(i)\hat{P}_i\right) - \log_2 \left(1 + T_1(i)T_2(i)P_i^{true}\right)\right] + \left[\delta_(y_i) - \delta(y_i)\right] \\
&= \log_2 \left(\frac{1 + T_1(i)T_2(i)\hat{P}_i}{1 + T_1(i)T_2(i)P_i^{true}}\right),
\end{aligned} \tag{12}$$

where $\hat{P}_i = \frac{N_i^{right}}{N_i^{sum}}$ is the Monte Carlo estimate of P_i^{true}. This error expression shows that the total error due to estimating the true probability P_i^{true} via Monte Carlo rollouts. Then, we derive an upper bound for the absolute error $|\mathcal{E}(y_i)|$. Using the triangle inequality, we have:

$$|\mathcal{E}(y_i)| \leq \left|\log_2 \left(\frac{1 + T_1(i)T_2(i)\hat{P}_i}{1 + T_1(i)T_2(i)P_i^{true}}\right)\right|. \tag{13}$$

Let $A = 1 + T_1(i)T_2(i)\hat{P}_i$ and $B = 1 + T_1(i)T_2(i)P_i^{true}$. The first term is $|\log_2(A) - \log_2(B)|$. Applying the Mean Value Theorem [35] shown in Equation 1, there exists ξ between A and B such that $\log_2(A) - \log_2(B) = (A - B)\frac{1}{\xi \ln 2}$. Thus,

$$|\log_2(A) - \log_2(B)| = |A - B|\frac{1}{|\xi| \ln 2}, \tag{14}$$

The difference $|A - B|$ is given by:

$$|A - B| = |(1 + T_1(i)T_2(i)\hat{P}_i) - (1 + T_1(i)T_2(i)P_i^{true})| = T_1(i)T_2(i)|\hat{P}_i - P_i^{true}|. \tag{15}$$

Since $T_1(i) = \frac{d_y - d_{\{y_1,\ldots,y_i\}}}{d_y}$ and $T_2(i) = \frac{d_y - d_{y_i}}{d_y}$ are ratios of lengths within long CoT, and assuming thoughts have non-zero length ($d_{y_i} > 0$) and the sequence does not cover the full length before thought n ($d_{\{y_1,\ldots,y_i\}} < d_y$ for $i < n$), we have $T_1(i) \in [0, 1)$ and $T_2(i) \in [0, 1)$. Also, $\hat{P}_i, P_i^{true} \in [0, 1]$. Therefore, $A, B \in [1, 2]$, which implies $\xi \in [1, 2]$ and $|\xi| \geq 1$. Substituting these into the inequality for the log term:

$$|\log_2(A) - \log_2(B)| \leq \frac{T_1(i)T_2(i)|\hat{P}_i - P_i^{true}|}{1 \cdot \ln 2} \leq \frac{|\hat{P}_i - P_i^{true}|}{\ln 2}, \tag{16}$$

where the term $|\hat{P}_i - P_i^{true}|$ represents the absolute error in the Monte Carlo estimation of the success probability. For a sufficiently large number of rollouts N_i^{sum}, this error is expected to be small and

Table 7: Training template for long thought reasoning model.

A conversation between the User and two Assistants. The User asks a question, and two Assistants collaborate to solve it. The first assistant tackles the hard steps, carefully thinks about the reasoning process in the mind, and then provides the reasoning process. The second assistant follows the provided reasoning process to complete the remaining straightforward steps to arrive at the final answer.

Two assistants switch roles to solve the question. The first assistant uses <think> to start its thought, and uses </think> to stop the thinking process. The second assistant user <answer> to complete the remaining steps, and use </answer> to stop the solving process. But if the second assistant finds the reasoning insufficient or encounters an error, they use </rethink> to request the first assistant to generate thoughts again.

The process is enclosed within <think>...</think>, <answer>...</rethink> and <answer>...</answer> tags, respectively, e.g., <think> the first assistant's reasoning process here </think> <answer> the second assistant's answer here </rethink> <think> the first assistant's reasoning process here </think> <answer> the second assistant's answer here </answer>.

You are the first assistant, you return reasoning process starts from <think>, and ends with </think>.

is bounded. By concentration inequalities (e.g., Hoeffding's), w.h.p., $|\hat{P}_i - P_i^{true}| \leq \epsilon$ where ϵ decreases with N_i^{sum}. Combining the bounds for both terms, the total error is bounded by:

$$|\mathcal{E}(y_i)| \leq \frac{|\hat{P}_i - P_i^{true}|}{\ln 2}. \tag{17}$$

Using the trivial bound ϵ for the probability error, we obtain an upper bound for the absolute error:

$$|\mathcal{E}(y_i)| \leq \frac{\epsilon}{\ln 2}. \tag{18}$$

This bound reveals that the error is mainly due to the probability nature of Monte Carlo rollout, and can be effectively reduced by increasing the sample size N_i^{sum}.

A.2 Prompt Template

This section introduces the design of the training template and the prompt used to enable automatic Long CoT chunking and short thought completion.

A.2.1 Training Template

As shown in Table 7 and Table 8, we display the training template for long-thought LLM and short-thought LLM in multi-turn RL training process. In general, these training template guide two LLMs follow tailored style to collaboratively reasoning.

A.2.2 Automatic Long Chain-of-Thought Chunking

Table A.2.2 shows the prompt template used for automatic long CoT chunking. We also provide a chunk example for a mathematical question "Given that $47^{-1} \equiv 51 \pmod{97}$, find $28^{-1} \pmod{97}$, as a residue modulo 97. (Give a number between 0 and 96, inclusive.)". As can be seen the Table A.2.2, we will first split Long CoT into multiple steps by '\n\n', and then we explicitly prompt LLM the chunk long CoT into multiple thoughts. In general, the chunked thought represent the key logical block in the overall long CoT reasoing process.

Table 8: Training template for the short thought reasoning model.

A conversation between the User and two Assistants. The User asks a question, and two Assistants collaborate to solve it. The first assistant tackles the hard steps, carefully thinks about the reasoning process in the mind, and then provides the reasoning process. The second assistant follows the provided reasoning process to complete the remaining straightforward steps to arrive at the final answer.

Two assistants switch roles to solve the question. The first assistant uses <think> to start its thought, and uses </think> to stop the thinking process. The second assistant user <answer> to complete the remaining steps, and use </answer> to stop the solving process. But if the second assistant finds the reasoning insufficient or encounters an error, they use </rethink> to request the first assistant to generate thoughts again.

The process is enclosed within <think>...</think>, <answer>...</rethink> and <answer>...</answer> tags, respectively, e.g., <think> the first assistant's reasoning process here </think> <answer> the second assistant's answer here </rethink> <think> the first assistant's reasoning process here </think> <answer> the second assistant's answer here </answer>.

You are the second assistant, you return completed remaining steps: (1) start from <answer>, and end with </rethink>, or (2) start from <answer>, and end with </answer>.

A2.2.2 Prompt for Automatic Long CoT Chunking

You are a professional math teacher. When solving math problems, you typically begin by understanding the problem, then break down the solution into several major steps, solving each part in sequence, arriving at the final result, and performing a second verification. Next, I will give you a student's solution process to a math problem. Based on this process, you need to reconstruct their general thinking process during problem-solving — for example, first understanding the problem, then decomposing it, and so on.
Specifically, please first read the problem and their solution. Then summarize the student's thinking process into distinct blocks such as "problem understanding" and so on. Record the start and end step index for each block. Return your answer in JSON format, where the key is "block 1", "block 2", etc., and the value includes the fields start, end, and block type. The start and end should refer to the index of each step, and block type should be a broad category (not too specific), such as "problem understanding".

Problem:
{Problem}

Solution:
{Long CoT}

Fill in start and end using step indices, and write block type. Return only the following JSON format without any explanations: 'block 1': 'start': '', 'end': '', 'block type': '', ...

A2.2.2 Example for Automatic Long CoT Chunking

Problem:
Given that $47^{-1} \equiv 51 \pmod{97}$, find $28^{-1} \pmod{97}$, as a residue modulo 97. (Give a number between 0 and 96, inclusive.)

Solution:
Step 0: So I have this problem here: given that $47^{-1} \equiv 51 \pmod{97}$, I need to find $28^{-1} \pmod{97}$, and it should be a number between 0 and 96, inclusive. Alright, let's see how to approach this.
Step 1: First off, $47^{-1} \pmod{97}$ means the multiplicative inverse of 47 modulo 97, which is a number that, when multiplied by 47, gives a result that's congruent to 1 modulo 97. They've already told me that $47^{-1} \equiv 51 \pmod{97}$, so that's a starting point.
Step 2: Now, I need to find the inverse of 28 modulo 97. That is, find a number x such that $28x \equiv 1 \pmod{97}$. I wonder if there's a way to relate this to the inverse of 47 that's already given.
Step 3: One thing I remember is that in modular arithmetic, if you have inverses of certain numbers, you might be able to combine them in useful ways. Maybe I can express 28 in terms of 47 or something related.
Step 4: Let me think about the relationship between 28 and 47 modulo 97. If I can find some expression that relates them, I might be able to use the known inverse of 47 to find the inverse of 28.
Step 5: Hmm, perhaps I can find a multiple of 47 that is congruent to 28 modulo 97. That is, find some integer k such that $47k \equiv 28 \pmod{97}$. If I can find such a k, then maybe I can manipulate that equation to find the inverse of 28.
Step 6: Let's try to solve $47k \equiv 28 \pmod{97}$. Since I know that $47^{-1} \equiv 51 \pmod{97}$, I can multiply both sides of the equation by 51 to solve for k:
Step 7: $k \equiv 28 \times 51 \pmod{97}$
Step 8: Let me calculate 28×51:
Step 9: $28 \times 50 = 1400$
Step 10: $28 \times 1 = 28$
Step 11: So, $1400 + 28 = 1428$
Step 12: Now, compute $1428 \pmod{97}$. To find this, I'll divide 1428 by 97 and find the remainder.
Step 13: First, $97 \times 10 = 970$, and $97 \times 4 = 388$, so total $970 + 388 = 1358$.
Step 14: Subtracting: $1428 - 1358 = 70$
Step 15: So, $1428 \equiv 70 \pmod{97}$, which means $k \equiv 70 \pmod{97}$.
...
Step 144: So, my final answer is that $28^{-1} \equiv 52 \pmod{97}$.
Step 145: Final answer: $\boxed{52}$

Chunked Thought:
{ "block 1": {"start": 0, "end": 5, "block type": "Question understanding"}, "block 2": {"start": 6, "end": 15, "block type": "Preliminary attempt"}, "block 3": {"start": 16, "end": 25, "block type": "Verify preliminary attempt"}, "block 4": {"start": 26, "end": 49, "block type": "Solve using extended Euclidean algorithm"}, "block 5": {"start": 50, "end": 69, "block type": "Try to solve using known inverse"}, "block 6": {"start": 70, "end": 81, "block type": "Verify answer"}, "block 7": {"start": 82, "end": 142, "block type": "Further attempts and verification"}, "block 8": {"start": 143, "end": 145, "block type": "Final conclusion"} }

A.2.3 Thought Completion

As shown in Table A.2.3, we present the prompt template used for thought completion, where a sequence of thoughts is completed based on the given thought type. The illustrative example in Table A.2.3 demonstrates that, during an incomplete solution process, we provide the corresponding thought types for the reasoning steps that need to be completed. Thought chunks that already include the reasoning process are regarded as long-thoughts, as previously discussed. We can see that, after thought completion the solution process will be merged by original long thought and completed short thought. These data will be further used for SFT dataset construction.

A2.2.3 Prompt for Thought Completion

Given a problem and its corresponding partially completed solution process, please complete the missing parts of the solution based on the specified thought identifiers. Return only the completed results in JSON format. Do not return the steps that have already been provided.
Problem:
{Problem}

Incompleted solution process:
{Incompleted solution process}

The parts you need to complete are the content fields corresponding to the following thoughts:
{Incompleted thought}

Note that the thought numbers are already specified—you must not split or modify them. Please strictly return the completions according to the given numbering, even if the required content for a single thought involves multiple steps.

You must return your answer in the following JSON format, and do not provide any explanation or content for other thoughts: {"Completed Steps": [{"thought": "string", "content": "string"}]}

A2.2.3 Example for Thought Completion

Problem:
Given that the arithmetic mean of two positive numbers a and b is 4, then the maximum value of the geometric mean of a and b is () A: 2 B: 4 C: 8 D: 16

Incompleted solution process:
{ "chunk 1": "Ok, so I have a question. This question is about the arithmetic mean and geometric mean. The question says that the arithmetic mean of two positive numbers a and b is 4. We need to find the maximum of their geometric means. The options are: a: 2, b: 4, c: 8, d: 16. First, I have to recall the definitions of the arithmetic mean and geometric mean. The arithmetic mean is $(a + b)/2$, and the geometric mean is $\sqrt{ab}$. The question says $(a + b)/2 = 4$, so we can conclude that $a + b = 8$.", "chunk 2": "Problem decomposition and preliminary solution", "chunk 3": "Wait, wait, I am a little confused, because there are larger numbers in the options, such as 8 and 16. Will there be a larger geometric mean? However, according to the inequality, the arithmetic mean is greater than or equal to the geometric mean, and only when $a = b$, the geometric mean is equal to the arithmetic mean. If a and b are not equal, the geometric mean will be less than the arithmetic mean. Assume that $a \neq b$, for example, $a = 5$, $b = 3$, then $a + b = 8$, and the geometric mean is $\sqrt{15} \approx 3.87$, which is indeed less than 4. For another example, if $a = 6$, $b = 2$, the geometric mean is $\sqrt{12} \approx 3.46$, which is still less than 4. It seems that the maximum value of the geometric mean is indeed 4.", "chunk 4": "Further verification", "chunk 5": "Conclusion" }

The parts you need to complete are the content fields corresponding to the following thoughts:
{"Problem decomposition and preliminary solution", "Further verification", "Conclusion"}

Completed thoughts:
[{ "thought": "Problem Analysis and Preliminary Solution", "content": "According to the property that the arithmetic mean is greater than or equal to the geometric mean, the geometric mean is equal to the arithmetic mean only when $a = b$. The condition given in the question is $(a + b)/2 = 4$, so $a + b = 8$. If $a = b$, then $a = b = 4$, and the geometric mean $\sqrt{ab} = \sqrt{4 \times 4} = 4$." }, { "thought": "Further Verification", "content": "Verify whether the geometric mean can be greater than 4 when $a \neq b$. Assuming $a > b$, and given that $a + b = 8$, we can set $a = 8 - b$. Then the geometric mean is $\sqrt{ab} = \sqrt{b(8 - b)}$. Since $b(8 - b)$ is a quadratic function that opens downward, its maximum value occurs when $b = 4$, that is, $a = b = 4$, and the geometric mean is 4. When $b \neq 4$, $b(8 - b)$ will be less than 16, and the geometric mean will also be less than 4. Therefore, the maximum value of the geometric mean is 4." }, { "thought": "Conclusion", "content": "According to the above analysis, the maximum value of the geometric mean is 4, and it can only be achieved when $a = b = 4$. Therefore, the correct answer is B: 4." }]

A.3 Details of Long⊗Short Training

We report the details of the Long⊗Short training process, including dataset construction, resource and costs and the visualization of RL training process of Long⊗Short. We begin by the dataset statistics, and introduce the detailed information about cold-start stage and multi-turn RL training stage.

A.3.1 Dataset Statistics

We report the dataset statistic in cold start stage and multi-turn RL training stage.

Quantitative Analysis Stage. In the quantitative analysis stage, we utilize chunked thoughts derived from long chains of thought (CoTs), which are distilled using DeepSeek-R1-Distill-Qwen7B and DeepSeek-R1-Distill-Llama8B. To provide a broader understanding of the structure and scale of the chunked data, we report detailed statistics in Table 9, including the minimum, average, and maximum number of thoughts per CoT, as well as the corresponding statistics for thought lengths (measured by token count). In general, the number of thoughts per CoT ranges from 2 to over 200, with average values around 10–16 depending on the dataset and model. Similarly, the length of individual thoughts

Table 9: Statistics of chunked long CoTs

Dataset	# num of thought			# length of thought		
	min	average	max	min	average	max
OpenMath-ThoughtChunk1.8K	2	15.93	211	8	799.3	25,188
Math-500-Thought-DeepSeek-R1-Distill-Qwen7B	2	10.67	133	13	929.26	17,597
GPQA-Dimaond-Thought-DeepSeek-R1-Distill-Qwen7B	4	14.95	132	9	1,484.60	36,842
Math-500-Thought-DeepSeek-R1-Distill-Llama8B	3	12.23	220	13	930.48	15,359
GPQA-Dimaond-Thought-DeepSeek-R1-Distill-Llama8B	4	16.12	130	15	1,630.85	22,751

varies significantly, from short spans of fewer than 10 tokens to long segments exceeding 30,000 tokens, highlighting the diverse granularity and complexity of reasoning captured in our chunking process.

Cold Start Stage. In the cold start stage, we use chunked thoughts obtained from 1.8K long CoTs extracted from the OpenMathInstruct dataset, as detailed in Section 5.1. Table 9 summarizes the key statistics of this chunked data, including the minimum, average, and maximum number of thoughts per CoT and the corresponding lengths of these thoughts. These statistics offer insight into the distribution and scale of reasoning units during the early stage, serving as a foundation for subsequent training or adaptation processes.

Multi-turn RL training Stage. For the multi-turn RL traing, we utilize MATH-ligtheval [44] (7.5K training samples and 5K validation samples in total) to conduct iterative self-evolution training.

A.3.2 Resources and Costs in Training

Quantitative Analysis Stage. To enable quantitative analysis of reasoning within long chains-of-thought (CoT), we perform 128 rollouts at each intermediate thought step. For the GPQA-Diamond dataset, this rollout process takes approximately 6 hours using 20 NVIDIA A100 GPUs. For the MATH 500 dataset, the same procedure requires around 10 hours on 20 NVIDIA A100 GPUs.

Cold Start Stage. In the SFT cold start stage, we first perform 8 rollouts at each thought step on 1.8K collected long CoTs using Qwen2.5-7B-Instruct, which takes roughly 10 hours on 40 NVIDIA A100 GPUs. Subsequently, we use Qwen2.5-72B-Instruct for thought completion as described in Section 4.1. This step takes approximately 2 hours on 40 NVIDIA A100 GPUs.

Multi-turn RL Training Stage. Each round of reinforcement learning (RL) training takes approximately 24 hours on 8 NVIDIA A100 GPUs.

A.4 Case Studies

To gain deeper insights into Long⊗Short, we conduct a case study using Long⊗Short-Qwen2.5-7B, comparing its behavior with DeepSeek-R1. We also provide a breakdown of the number of dialog turns involved in this multi-turn collaborative reasoning process. Finally, we analyze the emergence of the "aha" moment phenomenon during the model evolution process.

A.4.1 Comparison Between Long⊗Short and R1

The results are presented in Table A.4.1. As can be seen, Long⊗Short produces significantly more concise and focused reasoning traces compared to DeepSeek-R1. While R1 generates a long chain of self-reflections—some of which are tangential or redundant—Long⊗Short delivers a more structured and efficient explanation. Moreover, both models ultimately arrive at the correct answer, but their reasoning styles differ: R1 mimics internal dialogue and uncertainty resolution, whereas Long⊗Short is more deterministic and goal-oriented. This illustrates that Long⊗Short can retain reasoning effectiveness while reducing verbosity, suggesting a better alignment with user expectations in scenarios requiring clarity and precision.

Table 10: Detailed statistics of Long⊗Short thought reasoning paradigm.

Dataset	# num of turn			# average length of thought	
	min	average	max	long-thought	short-thought
Long⊗Short-Qwen2.5-7B					
Math 500	1	2.16	4	799	498
AIME 2024	1	2.47	5	1,473	653
AIME 2025	1	2.17	3	1,211	620
GPQA Diamond	1	2.13	3	1,425	859
AMC 2023	1	2.17	3	991	544
Long⊗Short-Llama3.1-8B					
Math 500	1	2.17	4	1,350	906
AIME 2024	1	2.25	4	1,340	1,224
AIME 2025	1	2.38	4	1,453	966
GPQA Diamond	1	2.11	3	1,407	859
AMC 2023	1	2.38	4	1,032	937

A.4.1 Case Study 1

Problem: 1+1=?

DeepSeek-R1:
<think>Okay, so the user is asking what 1 plus 1 equals. Let me think. Well, in basic arithmetic, when you add two numbers together, you're combining their values. So if I have one apple and someone gives me another apple, I have two apples. That makes sense. But wait, could there be any other interpretations here? Sometimes people use 1+1 in different contexts, like binary. In binary, 1 plus 1 would be 10, right? But the question doesn't specify the number system. Since it's a simple question without any context, probably the answer is expecting the standard base 10 result. So 1 + 1 should equal 2. But maybe I should double-check. Let me count on my fingers. One finger up, then add another finger. That's two fingers. Yep, that's right. So unless there's some trick or alternative meaning intended, the answer is 2.</think>
<answer>The result of 1 + 1 is **2**. This is a fundamental arithmetic operation in the base-10 number system. If you encounter this in a different context (e.g., binary, modular arithmetic, or symbolic logic), the answer might vary, but in standard mathematics, the answer is **2**.</answer>

Long⊗Short:
<think> Alright, so i've got this problem here: 1+1=? </think> <answer> The problem you've presented is a simple addition problem. When you add 1 and 1 together, the result is 2. So, the answer is 2. </answer>

A.4.2 Thought Length and Dialog Turn Anaylsis of Long⊗Short

Table 10 presents a quantitative analysis of the dialog turns and reasoning length under the Long⊗Short paradigm across various math and QA datasets.

Across both Qwen2.5-7B and Llama3.1-8B backbones, we observe that Long⊗Short typically completes reasoning within 2–3 turns, indicating a stable and efficient interaction pattern.

Interestingly, the average length gap between long-thought and short-thought responses suggests the paradigm's ability to modulate verbosity depending on the reasoning role: long-thoughts provide comprehensive problem-solving steps, while short-thoughts summarize or validate the solution. This supports the effectiveness of the Long⊗Short design in encouraging role-specialized reasoning styles.

A.4.3 Oha Moment of 'Overthinking' and 'Rethinking'

A.4.3 Case Study 2

Problem: Consider a rhombohedral crystal, with the interatomic distance of 10 Angstrom and the angles $\alpha = \beta = \gamma = 30^0$. What is the interplanar distance of the (111) plane of the crystal?

Long⊗Short:
<think> Alright, I've got this problem about a rhombohedral crystal. So, a rhombohedron is a type of prism where all six faces are rhombuses, and in this case, the interatomic distance is 10 Angstrom, and all the angles α, β, and γ are 30 degrees. I need to find the interplanar distance of the (111) plane in this crystal. First, I should recall what interplanar distance means. It's the distance between two parallel planes in a crystal lattice. In this case, I'm looking at the (111) plane, which is a specific plane in the rhombohedral lattice. </think> <answer> The interplanar distance in a rhombohedral crystal can be calculated using the formula for the interplanar distance in a lattice, which is

$$d_{hkl} = \frac{a}{\sqrt{h^2 + k^2 + l^2 - 2\cos(\alpha)hk - 2\cos(\beta)kl - 2\cos(\gamma)lh}},$$

where a is the lattice constant (10 Angstrom in this case), and h, k, l are the Miller indices of the plane (111 in this case). The angles α, β, γ are all 30 degrees. Plugging in the values, we get ... However, this does not match any of the given options. I will recheck the calculation. </rethink> <think> Maybe I need to consider the lattice constant differently. Maybe a is not just the interatomic distance, but something else. I need to understand the structure of the rhombohedral lattice. A rhombohedron can be thought of as a cube that's stretched along one axis. In this case, all angles are 30 degrees, which is different from a cube where angles are 90 degrees.... After some calculation, I find that the distance between adjacent atoms along a lattice vector in a rhombohedral lattice with angle 30 degrees is ...This seems incorrect because distance can't be negative. I need to check my signs. After correcting the signs, ... This is close to one of the options, but not exact. I need to verify the calculation. Wait, maybe I'm overthinking on this, the interplanar distance of the (111) plane is 9.54 Angstrom. </think> <answer> After rechecking the calculation, the correct value for the interplanar distance of the (111) plane in the rhombohedral crystal is approximately 9.54 Angstrom. This matches the candidate answer (1) 9.54 Angstrom. Therefore, the interplanar distance of the (111) plane is 9.54 Angstrom. </answer>

We report a oha moment case in Table A.4.3 for better understanding. As shown in Table 11, we analyze the emergence of oha moments self-reflective expressions such as 'Maybe I'm overthinking this' or 'Let me double-check'—across different training stages and datasets.

We summarize three key findings: *(1) Initial SFT Models Rarely Exhibit Oha Moments* In the early stage of training (i.e., SFT-only models, e.g., Long-r0⊗Short-r0 w/SFT), the frequency of oha moments is extremely low, often below 5% across all datasets. This indicates that SFT stage alone can only make LLM to learn the basic reasoning style, but the potential reasoning activity is still underactivated. *(2) Reinforcement Learning Significantly Enhances Oha Moment Emergence.* As the models undergo asynchronous multi-turn reinforcement learning, the frequency of oha moments steadily increases. For example, Long⊗Short-Llama3.1-8B reaches over 40% on AMIE 2024 and 2025 datasets by Round 3 and 4. This suggests that RL not only improves task performance but also encourages the model to engage in more reflective reasoning activity in such collaborative reasoning paradigm. *(3) Task Difficulty Correlates with Oha Moment Frequency.* We observe that oha moments are more frequent in datasets with higher complexity, such as AMIE 2024/2025 and GPQA Diamond. On these challenging benchmarks, oha moments appear in up to 40–43% of the model responses in later training rounds. In contrast, simpler datasets such as MATH500 show more modest increases, rarely exceeding 20%. This suggests that oha moments are not uniformly distributed but instead tend to occur when the model faces harder reasoning tasks, indicating that oha moments may serve as a proxy for perceived uncertainty or internal conflict during the long⊗short thought reasoning.

Table 11: The comparison of Long⊗Short across asynchronous evolution rounds reveals an increasing percentage (100%) of oha moments generated by the model.

Round#	MATH 500	AMIE 2024	AMIE 2025	GPQA Diamond	AMC 2023
Long⊗Short-Qwen2.5-7B					
Long-r0⊗Short-r0 w/SFT	3.33	6.67	3.33	1.20	2.25
Long-r1⊗Short-r0	6.67	13.33	6.67	3.33	5.23
Long-r2⊗Short-r0	15.64	36.67	16.67	9.53	15.53
Long-r2⊗Short-r1	16.67	33.33	11.53	7.65	22.50
Long-r3⊗Short-r1	16.50	40.33	10.54	12.50	25.25
Long-r3⊗Short-r2	14.50	42.33	16.67	14.26	24.47
Long-r4⊗Short-r2	16.60	40.00	20.00	12.62	25.00
Long⊗Short-Llama3.1-8B					
Long-r0⊗Short-r0 w/SFT	3.35	0	0	1.25	0
Long-r1⊗Short-r0	3.35	13.35	15.57	6.67	10.25
Long-r2⊗Short-r0	9.76	37.54	28.50	16.67	25.50
Long-r2⊗Short-r1	16.67	42.50	30.00	17.50	33.33
Long-r3⊗Short-r1	15.20	40.00	31.50	21.55	34.50
Long-r3⊗Short-r2	12.50	42.50	30.00	21.00	35.55
Long-r4⊗Short-r2	14.20	43.33	33.33	22.73	37.50

In summary, the progressive rise of oha moments through RL training—particularly on complex datasets—highlights the potential of collaborative and hybrid reasoning paradigm. These findings also suggest a promising direction for developing hybrid reasoning.